PNEUMONIAMNIST

CONFRONTO TRA SVM E CNN PER LA CLASSIFICAZIONE DI IMMAGINI MEDICHE

di **MATTEO LUGLI**

Computational & Statistical Learning

Laurea Magistrale in Matematica - Data Science

Università degli Studi di Modena e Reggio Emilia

OBIETTIVO

Confrontare due paradigmi di apprendimento:

- **SVM** → approccio statistico e regolarizzato
- **CNN** → approccio profondo ottimizzato con SGD e Adam

Collegare entrambe le parti del corso

Applicazione reale: diagnosi automatica di polmonite da radiografie

PNEUMONIAMNIST (MedMNIST V2)

- 5.856 immagini 28×28 grayscale di RX torace
- Classi → 0 = sano, 1 = polmonite
- Split → 4708 train, 524 val, 624 test (come da MedMNIST), dati bilanciati con leggera prevalenza di "pneumonia"
- Metriche
  - AUC (primaria) → capacità del modello di distinguere le 2 classi a tutte le possibili soglie
  - Accuracy → percentuale di predizioni corrette
  - F1 → misura quanto bene si riconosce la classe positiva
  - Brier score → se il modello assegna probabilità "coerenti" con la realtà, misura quanto sono calibrate le probabilità.

Formalizzazione

$$x_i \in \mathbb{R}^{\{28\times 28\}}, \qquad y_i \in \{0,1\}, \qquad f(x) \approx \mathbb{P}(Y = 1 \mid X = x)$$

TEORIA – SVM (1)

SVM per classificazione lineare

Obiettivo → trovare l'iperpiano di separazione $w^T x + b = 0$ che separa due classi con il **margine massimo**, cioè la distanza più grande possibile dai punti più vicini

$$\min \frac{1}{2} w^T w$$

$$\text{soggetto a } y_i(w^T \phi(x_i) + b) \geq 1, \text{ per } i = 1, \ldots, l$$

- distanza dei punti dall'iperpiano $\frac{1}{\|w\|}$
- massimizzare il margine ⇔ minimizzare $\frac{1}{2\|w\|^2}$

NON utilizzerò questo → PneumoniaMNIST è composto da immagini 28×28 in cui le classi non sono linearmente separabili nello spazio originale dei pixel.

TEORIA – SVM (2)

SVM per classificazione non lineare (soft-margin)

Obiettivo → risolvere il problema primale CL2

$$\min \frac{1}{2} w^T w + C \sum_{i=1}^{l} \xi_i$$

$$\text{soggetto a } y_i(w^T x_i + b) \geq 1 - \xi_i, \ \xi_i \geq 0$$

con $\xi_i = \max[0, 1 - y_i(w^T x_i - b)]$ variabili di **slack**, equivalente al problema duale DCL2

$$\max -\frac{1}{2} \sum_{i=1}^{l} \alpha_i \, \alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^{l} \alpha_i$$

$$\text{soggetto a } \sum_{i=1}^{l} \alpha_i y_i = 0 \,, \ \ 0 \leq \alpha_i \leq C$$

TEORIA – SVM (2)

SVM per classificazione non lineare (soft-margin)

Obiettivo → risolvere il problema primale CL2

parametro di regolarizzazione

$$\min \frac{1}{2} w^T w + C \sum_{i=1}^{l} \xi_i$$

permette errori di classificazione per pochi punti

soggetto a $y_i(w^T x_i + b) \geq 1 - \xi_i, \ \xi_i \geq 0$

$l(x, y, f(x)) = \max[0, 1 - f(x)] = \max[0, 1 - w^T x + b]$

con $\xi_i = \max[0, 1 - y_i(w^T x_i - b)]$ variabili di **slack**, equivalente al problema duale DCL2

$$\max -\frac{1}{2} \sum_{i=1}^{l} \alpha_i \, \alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^{l} \alpha_i$$

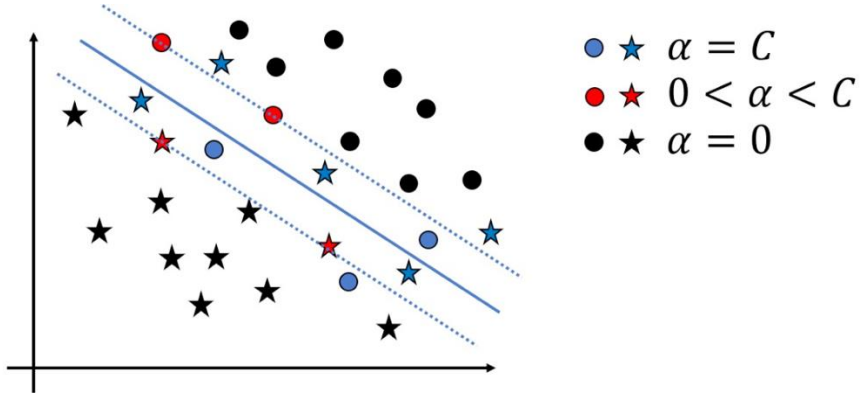
soggetto a $\sum_{i=1}^{l} \alpha_i y_i = 0 \,, \ \ 0 \leq \alpha_i \leq C$

TEORIA – SVM (3)

Determinato il vettore di moltiplicatori $\bar{\alpha}$, si ottiene

- $\bar{w} = \sum_{i=1}^{l} \bar{\alpha}_i y_i x_i$
- $\bar{b}$ e $\bar{\xi}$ con le condizioni di Kuhn–Tucker

$$\bar{\alpha}_i\left(y_i\left(\bar{w}^T x_i + \bar{b}\right) - 1 - \bar{\xi}_i\right) = 0 \quad \text{e} \quad (C - \bar{\alpha}_i)\bar{\xi}_i = 0$$



TEORIA – SVM (4)

Kernel trick

Teorema di Mercer permette di scrivere un kernel continuo come

$$k(x, z) = \langle \phi(x), \phi(z) \rangle_{\mathcal{F}}$$

con $\mathcal{F}$ spazio di feature e $\phi: X \to \mathcal{F}$, ovvero un kernel identifica una coppia $(\mathcal{F}, \phi)$ in modo che l'OSH in $\mathcal{F}$ corrisponde a una superficie di separazione non lineare in X

TEORIA – SVM (5)

Si ottiene la superficie di separazione computando $\bar{\alpha}_i$ che risolvono il DCL3

$$\max -\frac{1}{2}\sum_{i=1}^{l} \alpha_i \, \alpha_j y_i y_j k(x_i, x_j) + \sum_{i=1}^{l} \alpha_i$$

$$\text{soggetto a } \sum_{i=1}^{l} \alpha_i y_i = 0\,, \; 0 \leq \alpha_i \leq C$$

e lo step di classificazione si riduce a

$$f(x) = \text{sign}\left(\sum_{i=1}^{l} \bar{\alpha}_i \, y_i k(x_i, x_j) + \bar{b}\right)$$

Con SVM è essenziale costruire un buon spazio delle feature. Quale strategie di feature extraction?

TEORIA – PCA

Obiettivo → ridurre dimensionalità del dataset preservando il più possibile l'informazione (varianza) contenuta nei dati:

1. si centrano i dati
2. si calcola matrice di covarianza $\Sigma = \frac{1}{n} X^T X$
3. si estraggono autovettori (direzioni principali) e autovalori (varianza spiegata)
4. si ordina per varianza decrescente
5. si proiettano i dati sulle prime k componenti $Z = XW_k$

Implementazione lineare ⇒ veloce
Stabilizza SVM e riduce rischio overfitting

TEORIA – HOG

Obiettivo → descrivere un'immagine tramite la direzione dei gradienti locali, cioè come variano i bordi e le strutture nella scena:

- per ogni pixel si calcola

$$G_x = I(x+1, y) - I(x-1, y), \qquad G_y = I(x, y+1) - I(x, y-1), \qquad M = \sqrt{G_x^2 + G_y^2}, \qquad \theta = \arctan\frac{G_y}{G_x}$$

- immagine è suddivisa in celle e in ogni cella si costruisce istogramma di rotazioni
- celle sono raggruppate in blocchi sovrapposti e si normalizza il vettore del blocco
- gli istogrammi normalizzati dei blocchi sono concatenati lungo un vettore di feature

Histogram of Oriented Gradients
Estrae bordi, texture, direzioni del gradiente

TEORIA – DEEP LEARNING (1)

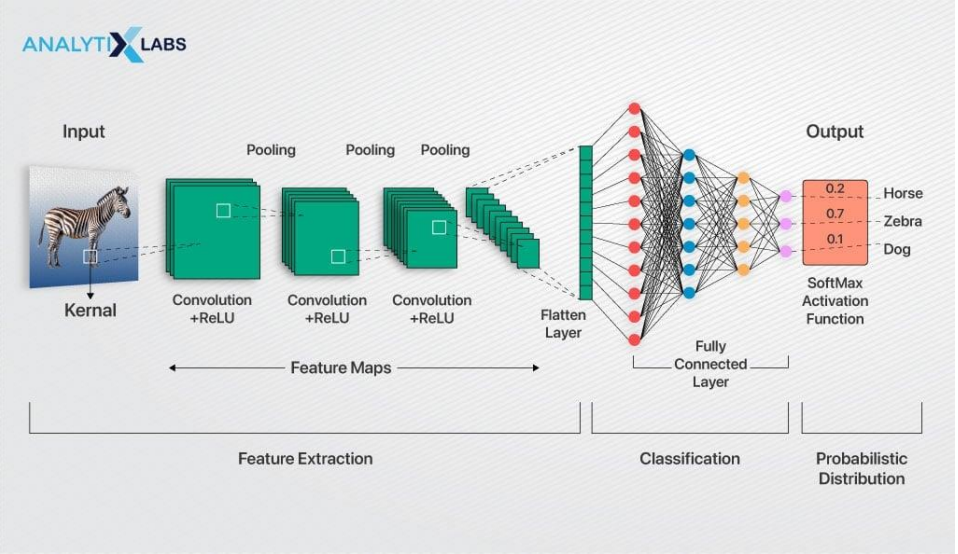
- Reti neurali artificiali e neuroni
- Reti unilayer (perceptron)
- Reti multilayer

TEORIA – DEEP LEARNING (2)

Immagini hanno struttura spaziale ⇒ Convolutional Neural Network (CNN)

Reti multilayer composte da

- strati convoluzionali (Conv2D)
- strati di attivazione (ReLU)
- strati di normalizzazione (BatchNorm)
- strati di pooling (MaxPool)
- strati fully-connected finali



TEORIA – DEEP LEARNING (3)

Sfruttano

- **convoluzioni** → filtri locali che scorrono sull'immagine, individuano pattern e producono feature map per indicare la presenza del pattern
- **parameter sharing** → filtro applicato ovunque nell'immagine (– parametri, + efficienza), favorisce generalizzazione (invarianza alla posizione) e riduce overfitting
- **pooling** → riduzione dimensione spaziale e rumore, invarianza a traslazioni locali e permettono costruzione di gerarchia di feature
- **strato fully connected** → combina tutte le feature estratte e produce l'output finale

TEORIA – DEEP LEARNING (4)

Componenti fondamentali

- BatchNorm
- ReLU (Rectified Linear Unit) → funzione di attivazione non lineare $\mathbf{ReLU}(x) = \mathbf{max}(0, x)$
- Max Pooling → riduce dimensione spaziale, mantiene attivazione massima nella regione (ritorna il massimo valore della porzione di immagine coperta dal kernel), riduce rumore, complessità e overfitting
- Dropout

TEORIA – DEEP LEARNING (5)

Regolarizzazione

Tecniche per correggere l'overfitting nei modelli di ML, per aumentare la generalizzabilità

- Weight decay L2 → $l'(\theta) = l(\theta) + \lambda\|\theta\|^2$
- Dropout → durante training, spegne casualmente una percentuale di neuroni, riducendo co-adattamento tra neuroni, efficace su dataset piccoli/medi
- Batch Normalization (BatchNorm) → normalizza le attivazioni all'interno del batch, stabilizza la distribuzione degli input ai layer successivi, permette learning rate più alti
- Early Stopping → monitora metrica di validazione

TEORIA – DEEP LEARNING (6)

Nel deep learning la funzione di perdita è troppo complessa e il dataset troppo grande per usare il gradiente esatto. Per questo si utilizza lo Stochastic Gradient Descent: una stima rumorosa ma molto più veloce del gradiente

TEORIA – SGM (1)

Il gradient descent classico usa tutto il dataset a ogni iterazione, troppo lento su dataset grandi. SGD invece usa un singolo esempio o un mini-batch

$$w_{t+1} = w_t - \eta \nabla \ell(x_i, y_i, w_t)$$

Se il learning rate (lr) soddisfa

$$\sum_t \eta_t = \infty, \sum_t \eta_t^2 < \infty$$

allora SGD converge in media al minimo.

TEORIA – SGM (2)

Metodo Stochastic Gradient (SG)

1. Scegliere w_0

2. for $k = 0,1, \ldots$ do
   - Genera una realizzazione di una v.a. ξ_{n_k}
   - Computa un vettore stocastico $g(w_k, \xi_{n_k})$
   - Scegli un learning rate $\alpha_k > 0$
   - $w_{k+1} \leftarrow w_k - \alpha g(w_k, \xi_{n_k})$

3. end for

Caratteristiche

- gradiente rumoroso, ma non distorto (stima corretta del gradiente atteso)
- permette molte iterazioni veloci
- introduce un rumore che aiuta a uscire da minimi piatti (implicit regularization)

Sensibilità al learning rate

- lr troppo grande → divergenza / oscillazioni
- lr troppo piccolo → convergenza lentissima
- lr fisso → ideale

TEORIA – VRM (1)

Momentum

1. Scegliere w_0, maxit e $\alpha, \beta \in [0,1)$

2. Inizializzare $m_0 \leftarrow 0,\ k \leftarrow 0$

3. for $k \in \{0,1,\dots,\text{maxit}\}$ do
   - $k \leftarrow k + 1$
   - $g_k \leftarrow \nabla f_{i_k}(w_{k-1})$
   - $m_k \leftarrow \beta m_k + g_k$
   - $w_k \leftarrow w_{k-1} - \alpha m_k$

4. end for

5. Result: w_k

AdaM

1. Scegliere $w_0, \text{maxit},$ e $\alpha, \hat{\varepsilon}, \beta_1, \beta_2 \in [0,1)$

2. Inizializzare $m_0 \leftarrow 0, v_0 \leftarrow 0, k \leftarrow 0$

3. for $k \in 0,1,\dots,$ do
   - $k \leftarrow k + 1$
   - $g_k \leftarrow \nabla f_{i_k}(w_{k+1})$
   - $m_k \leftarrow \beta_1 m_{k-1} + (1 - \beta_1) g_k$
   - $v_k \leftarrow \beta_2 v_{k-1} + (1 - \beta_2) g_k^2$
   - $\alpha_k = \alpha \frac{\sqrt{1-\beta_2^k}}{1-\beta_1^k}$
   - $w_k \leftarrow w_{k+1} - \alpha_k \frac{m_k}{\sqrt{v_k}+\hat{\varepsilon}}$

4. end for

TEORIA – VRM (2)

Momentum

- aggiunge memoria alla direzione del gradiente
- stabilizza e accelera
- smorza le oscillazioni nelle CNN
- converge più rapidamente di SG puro

AdaM

- learning rate adattivo per parametro
- ottima velocità nelle prime epoche
- molto stabile (soprattutto con dati rumorosi)
- converge più in fretta
- poco sensibile al seed

IN QUESTO PROGETTO...

Classificazione binaria (pneumonia vs. normal) sul dataset PneumoniaMNIST.

Confronto tra

$$\text{SVM} + \text{PCA} \;/\; \text{SVM} + \text{HOG} \quad \text{e} \quad \text{CNN compatta}$$

con: SGD + Momentum (variazione lr), Adam, esperimenti con e senza SEED

Analisi prestazioni, tempi di training, calibrazione (Brier) e stabilità dell'ottimizzazione.

Si dimostra che il feature learning (CNN) supera il feature engineering (HOG/PCA).

IN QUESTO PROGETTO...

Classificazione binaria (pneumonia vs. normal) sul dataset PneumoniaMNIST.

Confronto tra

$$\text{SVM} + \text{PCA} \;/\; \text{SVM} + \text{HOG} \quad \text{e} \quad \text{CNN compatta}$$

con: SGD + Momentum (variazione lr), Adam, esperimenti con e senza SEED

numero che inizializza il generatore di numeri casuali, serve a rendere riproducibili tutte le operazioni che contengono casualità

Analisi prestazioni, tempi di training, calibrazione (Brier) e stabilità dell'ottimizzazione.

Si dimostra che il feature learning (CNN) supera il feature engineering (HOG/PCA).

PIPELINE – SVM (1)

1. Preprocessing: immagini → vettori numerici

Le immagini 28×28 vengono flattened (784 valori)

Si applica StandardScaler per portare tutte le feature sulla stessa scala

PIPELINE – SVM (2)

2. Feature extraction

- PCA (95% varianza)
- HOG

PIPELINE – SVM (3)

3. Modello

- SVM non lineare con kernel RBF (Radial Basis Function)

$$k(x, x') = \exp(-\gamma \parallel x - x' \parallel^2)$$

- SVM cerca un confine di decisione che separa al meglio i due gruppi.

4. Ricerca dei parametri (GridSearchCV)

- cross-validation a 5 fold usando AUC come metrica
- ricerca della combinazione migliore di C (regolarizzazione) e γ (curvatura del kernel)

PIPELINE – SVM (4)

5. Validazione e Test

- Si scelgo il modello con miglior AUC sulla validation
- Si valuta sul test con: AUC (principale), Accuracy, F1, Brier Score.

6. Confronto finale

- Confronto tra **SVM+PCA** vs **SVM+HOG**.
- Con HOG ottieni il miglior baseline classico, perché cattura meglio le strutture polmonari.

PIPELINE – CNN (1)

1. Blocco: Conv2D (1 → 16, kernel 3×3, padding 1) → BatchNorm → ReLU → MaxPool (2×2)
2. Blocco: Conv2D (16 → 32, kernel 3×3) → BatchNorm → ReLU → MaxPool (2×2)
3. Blocco: Conv2D (32 → 64, kernel 3×3) → BatchNorm → ReLU
4. Global Average Pooling: tensore (64×7×7) → vettore dimensione 64
5. Dropout (p = 0.3)
6. Fully Connected layer: Linear (64 → 1)

Output: probabilità di polmonite

PIPELINE – CNN (1)

1. Blocco: Conv2D (1 → 16, kernel 3×3, padding 1) → BatchNorm → ReLU → MaxPool (2×2)
2. Blocco: Conv2D (16 → 32, kernel 3×3) → BatchNorm → ReLU → MaxPool (2×2)
3. Blocco: Conv2D (32 → 64, kernel 3×3) → BatchNorm → ReLU
4. Global Average Pooling: tensore (64×7×7) → vettore dimensione 64
5. Dropout (p = 0.3)
6. Fully Connected layer: Linear (64 → 1)

Output: probabilità di polmonite

MaxPooling prende solo il valore massimo di ogni feature map, mentre GAP rappresenta l'intera distribuzione delle feature

PIPELINE – CNN (2)

Perché queste scelte?

- 3 blocchi convoluzionali = 3 livelli di feature (pattern locali, bordi, gradienti; forme/tessiture polmonari; anomalie più astratte legate alla patologia)
- Canali 1→16→32→64: aumenta la profondità dei canali man mano che si riduce la risoluzione
- MaxPool nei primi due blocchi → riduce risoluzione, costo computazionale, overfitting; introduce invarianza a traslazioni
- GAP → trasforma tensore 64×7×7 in vettore di 64 numeri, riduce parametri

SETUP SPERIMENTALE (1)

Esperimenti comparativi

- SVM + PCA (varianza 95%)
- SVM + HOG (4×4)
- CNN con SGD (lr=0.05 e lr=0.01) (con e senza seed)
- CNN con Momentum (con e senza seed) → nella pratica non influenza il risultato finale nel progetto quanto AdaM
- CNN con AdaM (con e senza seed)

SETUP SPERIMENTALE (2)

Loss (SVM) $\rightarrow l(w; x, y) = \max\big(0, 1 - y(w^T x)\big)$

Loss (CNN) $\rightarrow$ Sigmoid + BCE integrata (BCEWithLogitsLoss)

$$l(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|})$$

- z = logit prodotto dalla CNN
- y = etichetta

ESPERIMENTO 1

Risultati SVM (PCA vs. HOG)

MODELLO	AUC	ACCURACY	F1	BRIER
SVM + PCA (95%)	~0.947	0.861	0.899	0.112
SVM + HOG (4×4)	~0.955	0.862	0.900	0.115

- SVM + PCA: baseline solido con dimensionalità ridotta.
- SVM + HOG: migliori prestazioni grazie a feature più informative.

ESPERIMENTO 1

Risultati CNN SGD vs. AdaM (no SEED)

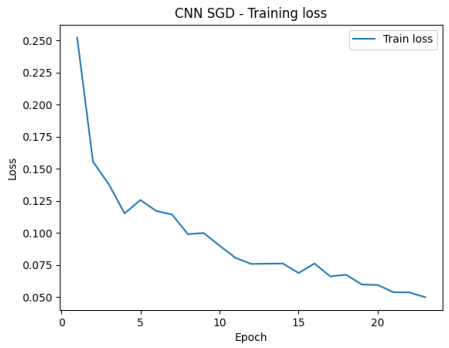
METODO	AUC	ACCURACY	F1	BRIER
CNN + SGD	~0.9558	0.705	0.809	0.237
CNN + AdaM	~0.9579	0.822	0.875	0.143

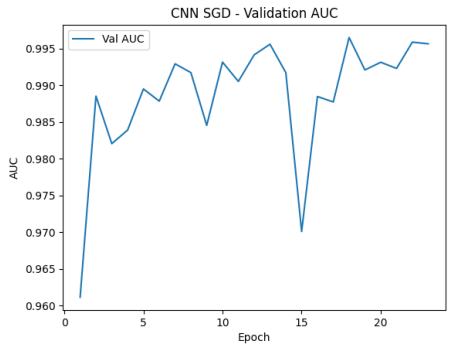
- AdaM converge in modo stabile anche senza SEED
- SGD mostra oscillazioni forti (lr=0.05)
- AdaM produce AUC più stabile

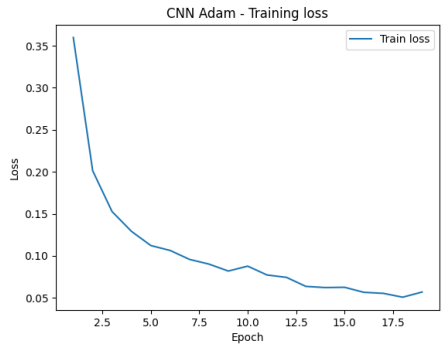
ESPERIMENTO 1

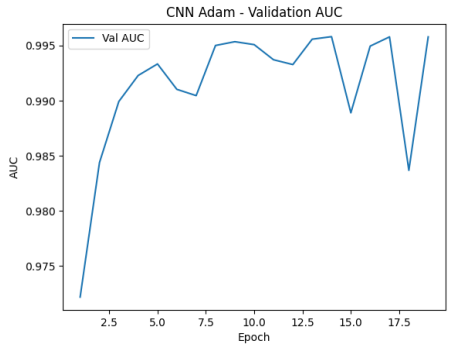
Risultati CNN SGD vs. AdaM (no SEED)

METODO	AUC	ACCURACY	F1	BRIER
CNN + SGD	~0.9558	0.705	0.809	0.237
CNN + AdaM	~0.9579	0.822	0.875	0.143









ESPERIMENTO 2

Risultati SVM (PCA vs. HOG) (con SEED)

MODELLO	AUC	ACCURACY	F1	BRIER
SVM + PCA (95%)	~0.947	0.861	0.899	0.112
SVM + HOG (4×4)	~0.955	0.862	0.900	0.115

SVM è stabile e deterministica, non esiste componente aleatoria nel training

ESPERIMENTO 2

Risultati CNN SGD vs. AdaM (con SEED)

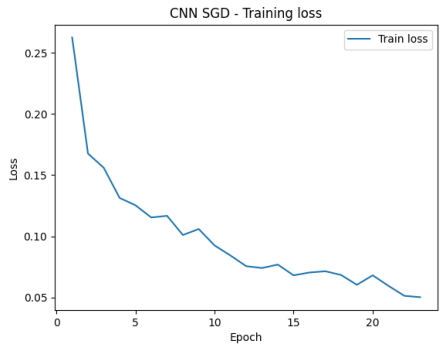
METODO	AUC	ACCURACY	F1	BRIER
CNN + SGD	~0.9660	0.888	0.907	0.078
CNN + AdaM	~0.9644	0.893	0.917	0.075

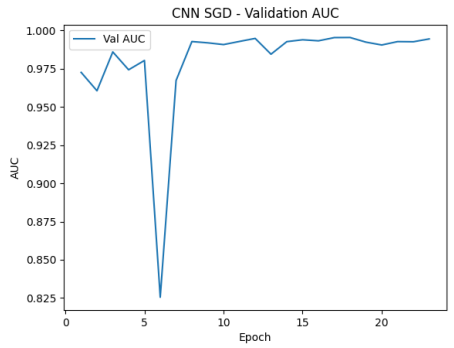
- SGD diventa molto più stabile con SEED fisso
- AdaM resta stabile
- Entrambi ottengono AUC e F1 molto alti

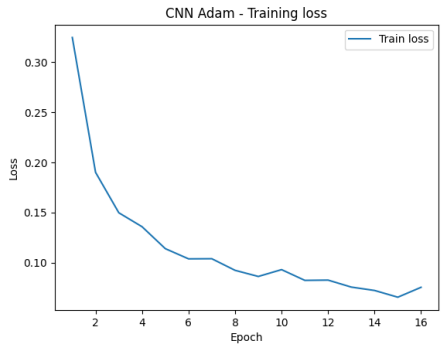
ESPERIMENTO 2

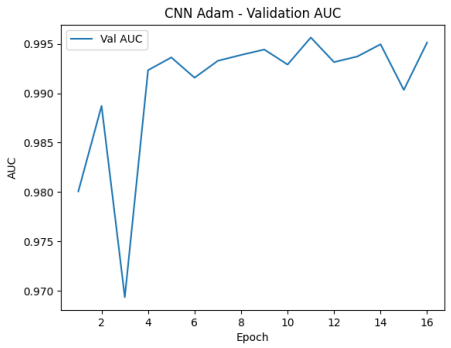
Risultati CNN SGD vs. AdaM (con SEED)

METODO	AUC	ACCURACY	F1	BRIER
CNN + SGD	~0.9660	0.888	0.907	0.078
CNN + AdaM	~0.9644	0.893	0.917	0.075









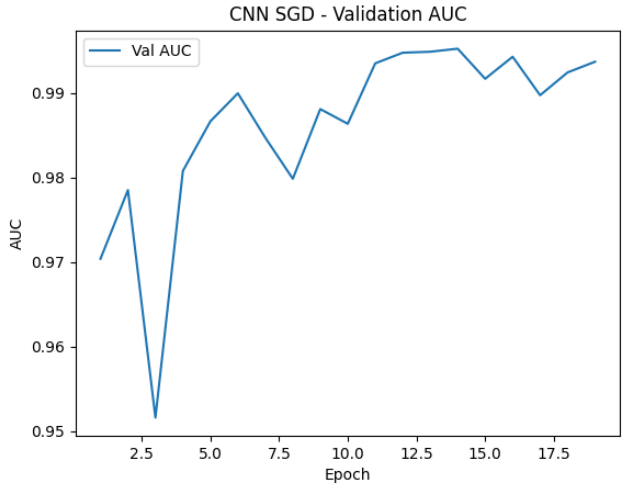
ESPERIMENTO 3

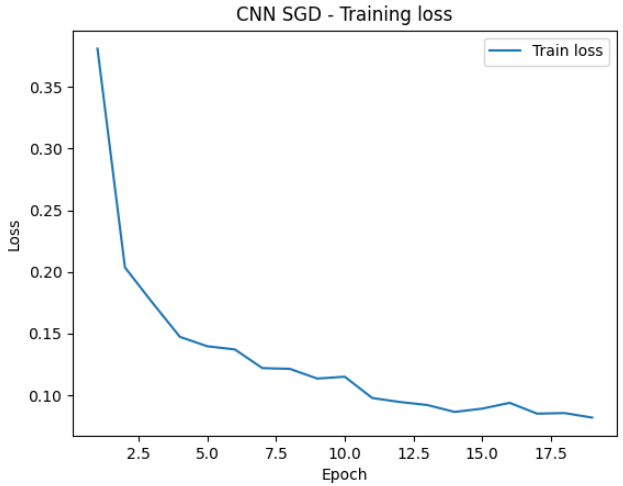
Risultati CNN SGD lr 0.01 (no SEED)

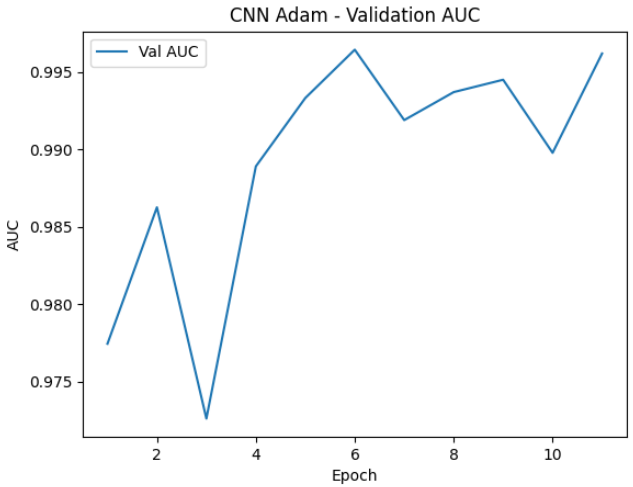
METODO	AUC	ACCURACY	F1	NOTE
lr = 0.05, no SEED (Exp 1)	0.9558	0.705	0.809	Instabile
lr = 0.01, no SEED (Exp 3)	0.9491	0.758	0.838	Più stabile
lr = 0.05, w/ SEED (Exp 2)	0.9660	0.888	0.907	Prestazioni ottime

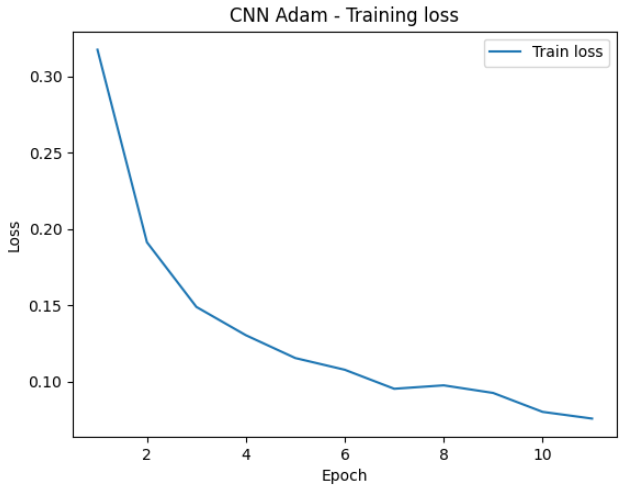
- Ridurre lr stabilizza fortemente lo SGD
- Oscillazioni meno «gravi»
- AUC in validazione molto più regolare

ESPERIMENTO 3









CONFRONTO FINALE

Modelli migliori

MODELLO	AUC	ACCURACY	F1	BRIER	TEMPO
SVM + HOG	0.955	0.862	0.900	0.115	~410s
CNN + SGD (seed ON)	0.966	0.888	0.907	0.078	~40s
CNN + AdaM (seed ON)	0.964	0.893	0.917	0.075	~25–40s

CONCLUSIONI

- SVM + HOG → ottimo baseline, stabile, deterministico
- CNN → migliori prestazioni grazie al feature learning
- SGD → molto sensibile a LR e seed, conferma teoria SGM
- Adam → convergenza più stabile e probabilità meglio calibrate
- CNN > SVM in AUC, F1 e Brier
- Deep learning più efficiente di SVM+HOG